

DOCUMENTO 06 — Arquitetura da Aplicação

Projeto: Lindvior

Versão: 1.0

Status: Aprovado

1. Objetivo

Este documento define a arquitetura da aplicação Lindvior.

Seu objetivo é estabelecer a organização estrutural do sistema, definindo a divisão em camadas, módulos, responsabilidades, fluxos internos e princípios arquiteturais que deverão ser seguidos durante todo o desenvolvimento da aplicação.

Enquanto:

- o Documento 01 define a visão geral do projeto;
- o Documento 02 define os requisitos funcionais;
- o Documento 03 define as regras de negócio;
- o Documento 04 define o modelo de domínio;
- o Documento 05 define o modelo de dados;

este documento define como esses elementos serão organizados na implementação da aplicação.

As definições apresentadas servirão como referência para toda a estrutura do código-fonte, reduzindo decisões arquiteturais durante o desenvolvimento e garantindo consistência entre os módulos do sistema.

2. Escopo

Este documento contempla:

- arquitetura em camadas;
- organização dos módulos da aplicação;
- responsabilidades de cada camada;

- fluxo interno das requisições;
- fluxo interno da simulação;
- separação de responsabilidades;
- princípios arquiteturais;
- organização dos pacotes;
- comunicação entre componentes;
- gerenciamento de transações;
- integração entre módulos;
- diretrizes para evolução arquitetural.

Não fazem parte deste documento:

- regras de negócio;
- estrutura do banco de dados;
- detalhes de implementação utilizando Spring Boot;
- configuração de dependências;
- infraestrutura de deploy;
- Docker;
- Kubernetes;
- Redis;
- Kafka;
- RabbitMQ;
- observabilidade;
- CI/CD.

Esses assuntos serão definidos em documentos específicos posteriormente.

3. Organização do Documento

Este documento está organizado nos seguintes capítulos.

4. Visão Geral da Arquitetura

Apresenta a arquitetura adotada pela aplicação e seus princípios gerais.

5. Arquitetura em Camadas

Define as camadas da aplicação e suas respectivas responsabilidades.

6. Organização dos Módulos

Define como as funcionalidades do sistema serão organizadas em módulos.

7. Fluxos da Aplicação

Define como as requisições percorrem a arquitetura da aplicação.

8. Motor da Simulação

Define a arquitetura responsável pela execução contínua da simulação.

9. Comunicação Interna

Define como os componentes da aplicação interagem entre si.

10. Gerenciamento de Transações

Define como as operações transacionais deverão ser organizadas na arquitetura.

11. Princípios Arquiteturais

Define os princípios que deverão orientar toda a implementação do sistema.

12. Evolução da Arquitetura

Define como a arquitetura poderá evoluir durante o desenvolvimento do projeto.

13. Considerações Gerais

Apresenta os princípios finais da arquitetura e sua integração com os demais documentos do Lindvior.

4. Visão Geral da Arquitetura

4.1 Objetivo

Este capítulo apresenta a arquitetura geral do Lindvior.

Seu objetivo é definir a estrutura de alto nível da aplicação, demonstrando como seus componentes serão organizados para atender aos requisitos funcionais, preservar as regras de negócio e permitir evolução contínua da plataforma.

A arquitetura adotada deverá servir como referência para toda a implementação do sistema, garantindo organização, baixo acoplamento, alta coesão e facilidade de manutenção. Ela deverá permanecer compatível com a visão do produto, os requisitos funcionais, o modelo de domínio e o modelo de dados definidos nos documentos anteriores.

4.2 Arquitetura Adotada

A versão 1.0 do Lindvior será desenvolvida utilizando uma arquitetura de **Monólito Modular**.

Toda a aplicação será executada em um único processo, compartilhando a mesma base de dados e a mesma infraestrutura de execução. Entretanto, sua organização interna deverá

ser estruturada em módulos independentes, reduzindo o acoplamento entre funcionalidades e facilitando futuras evoluções arquiteturais.

Cada módulo deverá possuir responsabilidades claramente definidas, evitando dependências desnecessárias e permitindo que novas funcionalidades sejam incorporadas sem comprometer a organização existente.

Embora a aplicação seja monolítica, sua estrutura deverá preparar o sistema para futuras extrações de serviços quando isso passar a representar um benefício real para o produto.

4.3 Características da Arquitetura

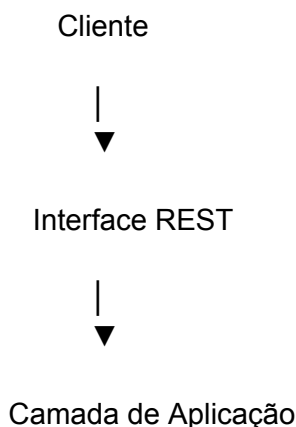
A arquitetura do Lindvior deverá possuir as seguintes características:

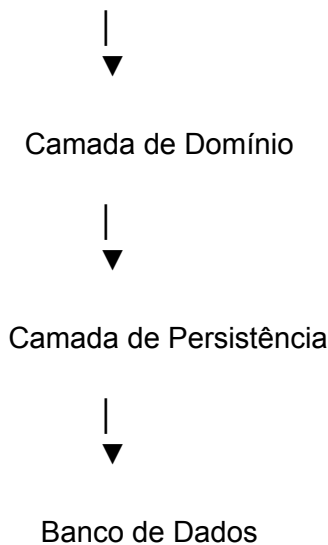
- execução em um único processo;
- banco de dados único;
- separação lógica por módulos;
- arquitetura em camadas;
- baixo acoplamento entre módulos;
- alta coesão interna;
- responsabilidades claramente definidas;
- regras de negócio concentradas na camada de domínio;
- independência entre regras de negócio e infraestrutura;
- facilidade para testes automatizados;
- evolução incremental da arquitetura.

Essas características deverão orientar todas as decisões estruturais tomadas durante o desenvolvimento do projeto.

4.4 Organização Geral

Em alto nível, a aplicação poderá ser representada pela seguinte estrutura:





Além do fluxo das requisições externas, a aplicação possuirá componentes internos responsáveis por executar tarefas contínuas, como o Motor de Simulação, que também utilizará as mesmas camadas arquiteturais para acessar o domínio e persistir informações.

4.5 Organização Modular

A arquitetura será organizada em módulos funcionais.

Cada módulo representará um conjunto coeso de funcionalidades pertencentes ao mesmo contexto de negócio.

Inicialmente, a aplicação será composta pelos seguintes módulos:

- Dashboard
- Administration
- Parking
- Simulation
- Reports
- Security
- Audit
- Shared

Esses módulos correspondem diretamente aos requisitos funcionais definidos para a versão 1.0 do produto e representam a divisão lógica da aplicação.

4.6 Independência entre Módulos

Os módulos deverão permanecer independentes sempre que possível.

Nenhum módulo deverá acessar diretamente detalhes internos de outro módulo.

Quando houver necessidade de colaboração entre módulos, essa comunicação deverá ocorrer exclusivamente por meio de contratos definidos pela própria arquitetura.

Essa separação tem como objetivos:

- reduzir o acoplamento;
 - facilitar manutenção;
 - permitir evolução independente;
 - simplificar testes;
 - preparar futuras evoluções arquiteturais.
-

4.7 Papel da Arquitetura

A arquitetura definida neste documento possui caráter estrutural.

Ela não define regras de negócio, requisitos funcionais ou detalhes de persistência.

Sua responsabilidade consiste exclusivamente em organizar os componentes da aplicação e estabelecer como eles colaboram para implementar os comportamentos descritos nos demais documentos do projeto.

Toda implementação do Lindvior deverá respeitar esta organização arquitetural, preservando a consistência entre estrutura, domínio e requisitos funcionais.

4.8 Camadas da Arquitetura

A arquitetura do Lindvior será organizada em camadas, cada uma responsável por um conjunto específico de responsabilidades.

Cada camada deverá conhecer apenas a camada imediatamente inferior, evitando dependências desnecessárias e preservando a organização da aplicação.

A estrutura geral será composta pelas seguintes camadas:

- Interface;
 - Aplicação;
 - Domínio;
 - Infraestrutura.
-

4.8.1 Camada de Interface

A Camada de Interface representa o ponto de entrada da aplicação.

Ela será responsável por receber requisições externas, validar informações básicas, encaminhar as operações para a camada de aplicação e devolver as respostas produzidas pelo sistema.

Responsabilidades:

- receber requisições HTTP;
- validar formatos básicos das entradas;
- converter requisições para objetos da aplicação;
- devolver respostas ao cliente;
- definir endpoints REST;
- não conter regras de negócio.

Componentes pertencentes à camada:

- Controllers;
- DTOs de Request;
- DTOs de Response;
- Handlers de exceção;
- Validações estruturais.

4.8.2 Camada de Aplicação

A Camada de Aplicação será responsável por coordenar os casos de uso da aplicação.

Ela não deverá implementar regras de negócio pertencentes ao domínio.

Sua função consiste em orquestrar operações, controlar transações, acionar os serviços necessários e coordenar o fluxo entre a interface, o domínio e a infraestrutura.

Responsabilidades:

- executar casos de uso;
- coordenar chamadas ao domínio;
- controlar transações;
- realizar orquestração entre componentes;
- aplicar políticas da aplicação;
- delegar decisões ao domínio.

Componentes pertencentes à camada:

- Services;
 - Facades (quando necessárias);
 - Casos de uso;
 - Orquestradores.
-

4.8.3 Camada de Domínio

A Camada de Domínio representa o núcleo do Lindvior.

Todas as regras de negócio deverão permanecer concentradas nesta camada.

Ela deverá permanecer independente de frameworks, banco de dados, APIs REST e tecnologias de infraestrutura.

Responsabilidades:

- representar o domínio;
- implementar regras de negócio;
- preservar invariantes;
- controlar o ciclo de vida das entidades;
- garantir consistência do modelo.

Componentes pertencentes à camada:

- Entidades;
 - Value Objects;
 - Enumerações;
 - Serviços de Domínio (quando necessários);
 - Interfaces pertencentes ao domínio.
-

4.8.4 Camada de Infraestrutura

A Camada de Infraestrutura será responsável por todas as integrações externas utilizadas pela aplicação.

Ela conterà as implementações técnicas necessárias para persistência, autenticação, mensageria, armazenamento de arquivos, observabilidade e demais recursos tecnológicos.

Responsabilidades:

- persistência dos dados;
- integração com banco de dados;
- autenticação;
- geração de tokens;
- comunicação externa;
- envio de mensagens;
- acesso a recursos de infraestrutura.

Componentes pertencentes à camada:

- Repositories;
- Implementações JPA;
- Configurações do Spring;

- Componentes de Segurança;
 - Clientes externos;
 - Implementações de mensageria;
 - Configurações de infraestrutura.
-

4.9 Fluxo Geral de uma Requisição

O fluxo de execução de uma operação deverá seguir a seguinte sequência:

Cliente



Controller



Service (Caso de Uso)



Domínio



Repository



Banco de Dados

Após a conclusão da operação, o resultado deverá percorrer o caminho inverso até retornar ao cliente.

Esse fluxo deverá ser seguido por todas as operações expostas pela API, garantindo uniformidade e previsibilidade durante toda a aplicação.

4.10 Fluxo Interno da Simulação

Além das requisições externas, o Lindvior possuirá componentes internos responsáveis pela execução contínua da simulação.

O Motor de Simulação deverá operar utilizando exatamente a mesma arquitetura utilizada pelas requisições REST.

Seu fluxo poderá ser representado da seguinte forma:

Simulation Engine



Caso de Uso



Domínio



Persistência

Dessa forma, as regras de negócio permanecerão centralizadas no domínio, independentemente de quem inicia a operação.

Isso garante que tanto uma requisição da API quanto uma execução automática da simulação utilizem exatamente as mesmas regras de negócio, evitando duplicação de lógica e preservando a consistência do sistema.

4.11 Princípios da Arquitetura

A arquitetura do Lindvior deverá seguir um conjunto de princípios que orientarão todas as decisões de implementação.

Esses princípios deverão ser respeitados independentemente da funcionalidade desenvolvida.

MA-001

A arquitetura deverá priorizar simplicidade.

Soluções arquiteturais somente deverão ser introduzidas quando resolverem um problema real da aplicação.

MA-002

Cada componente deverá possuir uma única responsabilidade claramente definida.

MA-003

As regras de negócio deverão permanecer concentradas na Camada de Domínio.

Nenhuma regra de negócio deverá ser implementada em Controllers, Repositories ou componentes de infraestrutura.

MA-004

Os componentes deverão possuir baixo acoplamento e alta coesão.

Mudanças em um módulo deverão produzir o menor impacto possível nos demais módulos.

MA-005

Toda dependência deverá apontar para camadas mais internas da aplicação.

Camadas inferiores nunca deverão depender de componentes pertencentes às camadas superiores.

MA-006

A arquitetura deverá facilitar testes automatizados.

Os componentes deverão ser organizados de forma a permitir testes unitários e testes de integração com o menor acoplamento possível.

MA-007

A arquitetura deverá ser orientada ao domínio.

As decisões de implementação deverão refletir o Modelo de Domínio definido no Documento 04 e não limitações impostas por frameworks ou tecnologias específicas.

MA-008

A evolução da arquitetura deverá ocorrer de forma incremental.

Novos componentes arquiteturais somente deverão ser adicionados quando houver necessidade comprovada.

MA-009

Toda funcionalidade deverá seguir a mesma organização arquitetural.

Não deverão existir exceções estruturais entre módulos semelhantes.

MA-010

A arquitetura deverá permanecer preparada para futuras evoluções sem exigir reestruturações significativas do código-fonte.

4.12 Benefícios da Arquitetura

A arquitetura definida neste documento proporciona os seguintes benefícios para o desenvolvimento do Lindvior:

- organização clara das responsabilidades;
- redução do acoplamento entre componentes;
- maior facilidade para manutenção;
- melhor reutilização de código;
- maior facilidade para implementação de testes automatizados;
- evolução incremental da aplicação;
- preparação para futuras expansões arquiteturais;
- maior previsibilidade durante o desenvolvimento.

Esses benefícios deverão ser preservados ao longo de toda a evolução do projeto.

5. Arquitetura em Camadas

5.1 Objetivo

Este capítulo define a arquitetura em camadas adotada pelo Lindvior.

Seu objetivo é estabelecer a divisão de responsabilidades da aplicação, determinando quais componentes pertencem a cada camada, quais responsabilidades cada uma deverá assumir e como ocorrerá a comunicação entre elas.

A utilização de camadas visa reduzir o acoplamento, aumentar a organização do código-fonte e facilitar a evolução da aplicação.

5.2 Visão Geral

A aplicação será organizada em quatro camadas principais.

Interface	
Aplicação	
Domínio	
Infraestrutura	

Cada camada possuirá responsabilidades específicas e bem delimitadas.

Nenhuma camada deverá assumir responsabilidades pertencentes a outra.

5.3 Camada de Interface

Objetivo

Representar o ponto de entrada da aplicação.

Será responsável por receber requisições externas, realizar validações estruturais, encaminhar os casos de uso para a camada de aplicação e retornar as respostas ao cliente.

Responsabilidades

- receber requisições HTTP;

- expor endpoints REST;
 - realizar validações estruturais;
 - converter Request DTO para objetos da aplicação;
 - converter objetos da aplicação para Response DTO;
 - retornar códigos HTTP adequados;
 - tratar exceções da API.
-

Componentes

- Controllers
 - Request DTOs
 - Response DTOs
 - Mappers
 - Global Exception Handler
 - Validadores de entrada
-

O que NÃO pertence à Interface

- regras de negócio;
 - acesso ao banco de dados;
 - consultas JPA;
 - decisões do domínio;
 - controle de transações.
-

5.4 Camada de Aplicação

Objetivo

Coordenar os casos de uso da aplicação.

Esta camada será responsável por orquestrar o fluxo das operações, delegando todas as decisões de negócio ao domínio e todas as operações técnicas à infraestrutura.

Responsabilidades

- executar casos de uso;
 - coordenar chamadas entre componentes;
 - iniciar e finalizar transações;
 - controlar o fluxo da operação;
 - orquestrar múltiplos serviços quando necessário.
-

Componentes

- Services
 - Use Cases
 - Facades
 - Orquestradores
 - Mappers (quando necessários)
-

O que NÃO pertence à Aplicação

- regras de negócio complexas;
 - consultas SQL;
 - acesso direto ao banco;
 - detalhes de infraestrutura.
-

5.5 Camada de Domínio

Objetivo

Representar o núcleo da aplicação.

Toda regra de negócio deverá estar concentrada nesta camada.

O domínio deverá permanecer independente de frameworks, banco de dados e tecnologias externas.

Responsabilidades

- implementar regras de negócio;
- preservar invariantes;

- validar estados do domínio;
 - representar entidades;
 - representar objetos de valor;
 - representar enumerações;
 - definir contratos pertencentes ao domínio.
-

Componentes

- Entities;
 - Value Objects;
 - Enums;
 - Domain Services;
 - Interfaces do domínio.
-

O que NÃO pertence ao Domínio

- Controllers;
 - Repositories JPA;
 - Spring Boot;
 - autenticação;
 - geração de JWT;
 - banco de dados;
 - APIs externas.
-

5.6 Camada de Infraestrutura

Objetivo

Implementar todas as dependências externas utilizadas pela aplicação.

Esta camada será responsável por transformar os contratos definidos nas camadas superiores em implementações concretas.

Responsabilidades

- persistência dos dados;

- acesso ao banco de dados;
 - autenticação;
 - geração e validação de JWT;
 - envio de mensagens;
 - acesso a arquivos;
 - configurações do framework;
 - integrações externas.
-

Componentes

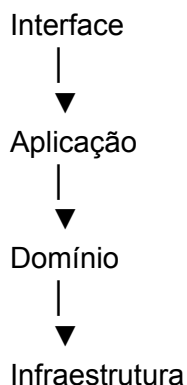
- Repositories
 - Implementações JPA
 - Specifications
 - Configurações do Spring
 - Componentes de Segurança
 - Clientes HTTP
 - Componentes de Mensageria
 - Configurações de Infraestrutura
-

O que NÃO pertence à Infraestrutura

- regras de negócio;
 - decisões do domínio;
 - orquestração de casos de uso.
-

5.7 Comunicação entre as Camadas

A comunicação entre as camadas deverá ocorrer exclusivamente no sentido descendente.



Nenhuma camada poderá acessar diretamente uma camada superior.

Essa restrição reduz o acoplamento e preserva a organização arquitetural da aplicação.

5.8 Dependências Permitidas

Camada	Pode depender de
Interface	Aplicação
Aplicação	Domínio e Infraestrutura
Domínio	Apenas do próprio Domínio
Infraestrutura	Domínio

Essas dependências deverão ser respeitadas durante toda a implementação do projeto.

5.9 Princípios das Camadas

MC-001

Cada camada deverá possuir responsabilidade única e claramente definida.

MC-002

Nenhuma camada deverá executar responsabilidades pertencentes a outra camada.

MC-003

Toda regra de negócio deverá permanecer concentrada no domínio.

MC-004

A infraestrutura deverá implementar apenas aspectos técnicos da aplicação.

MC-005

A camada de aplicação deverá atuar exclusivamente como coordenadora dos casos de uso.

MC-006

A interface deverá permanecer responsável apenas pela comunicação com clientes externos.

MC-007

Toda comunicação entre camadas deverá respeitar as dependências definidas neste documento.

MC-008

A organização em camadas deverá permanecer uniforme em todos os módulos da aplicação.

6. Organização dos Módulos

6.1 Objetivo

Este capítulo define como as funcionalidades do Lindvior serão organizadas em módulos.

Seu objetivo é dividir a aplicação em conjuntos coesos de responsabilidades, reduzindo o acoplamento entre funcionalidades e facilitando a manutenção, os testes e a evolução do sistema.

Os módulos representam uma organização lógica da aplicação e não deverão ser interpretados como aplicações independentes.

6.2 Organização Geral

A versão 1.0 do Lindvior será composta pelos seguintes módulos:

Lindvior

—	Dashboard
—	Administration
—	Parking
—	Simulation
—	Reports
—	Security
—	Audit
—	Shared

Cada módulo deverá possuir responsabilidades claramente definidas e comunicar-se com os demais apenas quando necessário.

6.3 Módulo Dashboard

Objetivo

Centralizar todas as funcionalidades responsáveis pela visualização do estado atual da operação do estacionamento.

Responsabilidades

- indicadores operacionais;
 - ocupação do estacionamento;
 - ocupação por setor;
 - disponibilidade de vagas;
 - estatísticas em tempo real;
 - informações utilizadas pela interface principal.
-

Componentes

- Dashboard Services;
 - Dashboard Controllers;
 - Dashboard DTOs;
 - consultas específicas do dashboard.
-

6.4 Módulo Administration

Objetivo

Centralizar todas as funcionalidades administrativas da aplicação.

Responsabilidades

- gerenciamento do estacionamento;
 - gerenciamento dos usuários;
 - parametrizações administrativas.
-

Componentes

- Controllers administrativos;
 - Services administrativos;
 - DTOs administrativos;
 - validações administrativas.
-

6.5 Módulo Parking

Objetivo

Representar o núcleo operacional do estacionamento.

Este módulo será responsável pelas operações relacionadas à utilização das vagas e ao gerenciamento das sessões de estacionamento.

Responsabilidades

- gerenciamento das vagas;
- gerenciamento das sessões;
- controle da ocupação;
- consulta das vagas;
- consulta das sessões.

Componentes

- Parking Services;
 - Parking Controllers;
 - regras operacionais;
 - consultas do estacionamento.
-

6.6 Módulo Simulation

Objetivo

Executar continuamente a simulação do estacionamento.

Este módulo será responsável por gerar o comportamento da população simulada e produzir os eventos que representam a operação do estacionamento.

Responsabilidades

- geração de veículos;
 - decisões da simulação;
 - entrada de veículos;
 - saída de veículos;
 - atualização da ocupação;
 - execução contínua da simulação.
-

Componentes

- Simulation Engine;
 - Simulation Scheduler;
 - Simulation Services;
 - geradores da simulação.
-

6.7 Módulo Reports

Objetivo

Centralizar todas as consultas analíticas da aplicação.

Responsabilidades

- relatórios operacionais;
 - estatísticas;
 - consultas históricas;
 - exportação de informações;
 - indicadores administrativos.
-

Componentes

- Report Controllers;
 - Report Services;
 - consultas específicas;
 - exportadores.
-

6.8 Módulo Security

Objetivo

Centralizar todos os mecanismos responsáveis pela autenticação e autorização da aplicação.

Responsabilidades

- autenticação;
 - autorização;
 - geração de JWT;
 - validação de JWT;
 - controle de permissões;
 - configuração da segurança.
-

Componentes

- Security Config;
 - Jwt Service;
 - Authentication Service;
 - Filters;
 - UserDetailsService.
-

6.9 Módulo Audit

Objetivo

Centralizar funcionalidades relacionadas ao acompanhamento administrativo da aplicação.

Na versão 1.0, este módulo será responsável apenas pelas estruturas necessárias para suportar futuras funcionalidades de auditoria.

Responsabilidades

- preparação para auditoria;
 - organização dos componentes relacionados;
 - evolução futura do módulo.
-

Componentes

- estruturas de auditoria;
 - serviços futuros.
-

6.10 Módulo Shared

Objetivo

Concentrar componentes reutilizados por toda a aplicação.

Este módulo não deverá conter regras de negócio.

Responsabilidades

- classes utilitárias;
 - constantes compartilhadas;
 - exceções comuns;
 - componentes reutilizáveis;
 - conversores;
 - abstrações comuns.
-

Componentes

- Exceptions;
 - Utils;
 - Constants;
 - Common Validators;
 - Classes compartilhadas.
-

6.11 Comunicação entre os Módulos

Os módulos deverão permanecer o mais independentes possível.

A comunicação entre eles deverá ocorrer apenas quando uma funcionalidade depender diretamente de outra.

Nenhum módulo deverá acessar diretamente detalhes internos de implementação de outro módulo.

Sempre que possível, a comunicação deverá ocorrer por meio da camada de aplicação.

6.12 Princípios da Modularização

MM-001

Cada módulo deverá possuir responsabilidade claramente definida.

MM-002

Os módulos deverão apresentar alta coesão e baixo acoplamento.

MM-003

Funcionalidades pertencentes ao mesmo contexto de negócio deverão permanecer no mesmo módulo.

MM-004

Componentes compartilhados deverão ser concentrados exclusivamente no módulo **Shared**.

MM-005

O crescimento da aplicação não deverá comprometer a organização modular definida neste documento.

MM-006

Novos módulos somente poderão ser criados quando representarem novos contextos funcionais claramente identificados.

MM-007

A organização modular deverá facilitar futuras evoluções arquiteturais, incluindo eventual extração de serviços independentes.

6.13 Organização dos Pacotes

|— config

|— shared

|— security

|— dashboard

|— parking

| |— controller

| |— service

| |— repository

| |— specification

| |— domain

| |— dto

| |— mapper

| |— exception

|— simulation

|— reports

└— audit

A estrutura apresentada representa a organização lógica da aplicação. Novos pacotes poderão ser adicionados quando necessários, desde que preservem a organização modular definida neste documento.

7. Fluxos da Aplicação

7.1 Objetivo

Este capítulo define como as operações percorrem a arquitetura do Lindvior.

Seu objetivo é estabelecer um fluxo único para execução dos casos de uso, garantindo que todas as funcionalidades da aplicação sigam a mesma sequência de processamento, independentemente de serem iniciadas por uma requisição externa ou por componentes internos da simulação.

7.2 Fluxo de uma Requisição HTTP

Toda requisição recebida pela API deverá seguir o fluxo arquitetural definido neste documento.

Cliente

|



Controller

|



Service (Caso de Uso)

|



Domínio

|



Repository

|



Banco de Dados

Após a conclusão da operação, o resultado deverá percorrer o caminho inverso até retornar ao cliente.

7.3 Etapas do Fluxo

1. Recepção da Requisição

A camada de Interface recebe a requisição HTTP.

Responsabilidades:

- identificar o endpoint;
 - validar estrutura dos dados;
 - converter Request DTO.
-

2. Execução do Caso de Uso

A Camada de Aplicação recebe a operação.

Responsabilidades:

- iniciar o caso de uso;
 - coordenar a operação;
 - controlar transações;
 - acionar o domínio.
-

3. Processamento do Domínio

O Domínio executa todas as regras de negócio necessárias.

Responsabilidades:

- validar regras;
 - verificar estados;
 - alterar entidades;
 - garantir consistência.
-

4. Persistência

A Infraestrutura realiza o acesso ao banco de dados.

Responsabilidades:

- consultar registros;
 - persistir alterações;
 - executar operações transacionais.
-

5. Construção da Resposta

Após a conclusão da operação, o resultado retorna até a Interface.

Responsabilidades:

- converter Response DTO;
 - definir código HTTP;
 - devolver resposta ao cliente.
-

7.4 Fluxo do Motor de Simulação

O Motor de Simulação deverá utilizar exatamente a mesma arquitetura utilizada pelas requisições REST.

Simulation Scheduler

|



Simulation Engine

|



Service

|



Domínio

|



Repository

|



Banco de Dados

O motor da simulação não deverá acessar diretamente a camada de persistência.

Toda alteração deverá passar pelas mesmas regras de negócio utilizadas pela API.

7.5 Fluxo de Autenticação

As operações protegidas deverão seguir o fluxo abaixo.

Cliente

|



JWT Filter

|



Spring Security

|



Controller

|



Service

|



Domínio

Caso a autenticação falhe, a requisição deverá ser encerrada antes de atingir a camada de Interface.

7.6 Fluxo de Tratamento de Erros

Durante qualquer etapa da execução poderão ocorrer exceções.

O tratamento deverá seguir o seguinte fluxo.

Exceção

|



Global Exception Handler

|



Response DTO

|



Cliente

As exceções deverão ser convertidas em respostas padronizadas da API.

7.7 Fluxo das Transações

Operações que modificarem o estado da aplicação deverão ser executadas dentro de uma transação.

Service

|

BEGIN

|

Domínio

|

Repository

|

COMMIT / ROLLBACK

A camada de aplicação será responsável pelo controle da transação.

7.8 Fluxo de Leitura

Operações exclusivamente de consulta seguirão um fluxo simplificado.

Cliente

|

Controller

|

Service

|

Repository

|

Banco

Como não existe alteração do estado do domínio, nenhuma transação de escrita será necessária.

7.9 Uniformidade dos Fluxos

Todos os módulos da aplicação deverão seguir os mesmos padrões de execução.

Independentemente da funcionalidade desenvolvida, o fluxo arquitetural deverá permanecer uniforme.

Essa uniformidade reduz o acoplamento, simplifica a manutenção e facilita o entendimento do sistema.

7.10 Princípios dos Fluxos

MF-001

Toda operação deverá percorrer as camadas definidas pela arquitetura.

MF-002

Nenhuma operação poderá acessar diretamente a camada de persistência ignorando o domínio.

MF-003

Toda alteração de estado deverá respeitar as regras de negócio antes da persistência.

MF-004

O Motor de Simulação deverá utilizar os mesmos casos de uso utilizados pela API.

MF-005

As exceções deverão ser tratadas de forma centralizada.

MF-006

Os fluxos da aplicação deverão permanecer consistentes em todos os módulos do sistema.

7.11 Exemplo de Fluxo Completo

POST /api/v1/parking-spots

|

Controller

↓

Mapper

↓

Service

↓

Domínio

↓

Repository



Banco de Dados



Mapper



Response DTO

Este exemplo representa o fluxo completo de um caso de uso típico da aplicação. Todos os demais casos de uso deverão seguir a mesma organização arquitetural, respeitando as camadas definidas neste documento.

8. Motor da Simulação

8.1 Objetivo

Este capítulo define a arquitetura do Motor de Simulação do Lindvior.

Seu objetivo é estabelecer como a simulação será organizada internamente, definindo seus componentes, responsabilidades e fluxo de execução.

O Motor de Simulação será responsável por reproduzir continuamente a operação do estacionamento, gerando os eventos que alimentam a aplicação e produzem os dados persistidos no banco de dados.

8.2 Visão Geral

O Motor de Simulação será um componente interno da aplicação.

Ele executará continuamente durante o funcionamento do sistema, simulando o comportamento dos veículos e produzindo eventos de entrada e saída do estacionamento.

Sua arquitetura poderá ser representada da seguinte forma.

Simulation Scheduler

|



Simulation Engine

|



Decision Engine

|



Application Services

|



Domínio

|



Persistência

Cada componente possuirá responsabilidade única e bem definida.

8.3 Componentes do Motor

Simulation Scheduler

Responsável por controlar o ciclo contínuo da simulação.

Responsabilidades:

- iniciar a simulação;
 - controlar a frequência de execução;
 - disparar novos ciclos;
 - manter a execução contínua.
-

Simulation Engine

Representa o núcleo operacional da simulação.

Responsabilidades:

- coordenar cada ciclo;
 - solicitar decisões;
 - executar operações da simulação;
 - controlar o fluxo interno.
-

Decision Engine

Responsável por determinar quais eventos ocorrerão em cada ciclo da simulação.

Exemplos:

- chegada de veículos;
- saída de veículos;
- seleção de veículos;
- escolha das vagas;
- decisões probabilísticas.

Este componente não deverá persistir dados.

Sua responsabilidade consiste apenas em produzir decisões.

Application Services

Após uma decisão ser produzida, os mesmos casos de uso utilizados pela API deverão ser executados.

Isso garante que a simulação utilize exatamente as mesmas regras de negócio disponíveis para operações externas.

8.4 Ciclo da Simulação

Cada ciclo da simulação deverá seguir a seguinte sequência.

Início do Ciclo

|

Gerar decisões

|

Executar casos de uso

|

Aplicar regras de negócio

|

Persistir alterações

|

Encerrar ciclo

|

Próximo ciclo

Após o encerramento de um ciclo, um novo ciclo deverá ser iniciado automaticamente enquanto a simulação permanecer ativa.

8.5 Integração com a Arquitetura

O Motor de Simulação não constitui uma arquitetura paralela.

Ele utilizará exatamente os mesmos componentes empregados pelas requisições REST.

Assim, sempre que uma decisão precisar alterar o estado do sistema, ela deverá utilizar:

- Services;
- Domínio;
- Repositories;
- Transações.

Nenhuma regra de negócio poderá existir exclusivamente no Motor de Simulação.

8.6 Estado da Simulação

O estado atual da simulação será representado exclusivamente pelos dados persistidos pela aplicação.

O Motor de Simulação não deverá manter estados paralelos que possam divergir das informações armazenadas no banco de dados.

Sempre que necessário, o estado atual deverá ser obtido por meio dos componentes responsáveis pela persistência.

8.7 Escalabilidade

O Motor de Simulação deverá ser desenvolvido de forma modular.

Novos comportamentos poderão ser incorporados futuramente, tais como:

- eventos especiais;
- sazonalidade;
- aceleração do tempo;
- perfis avançados de comportamento;
- múltiplas estratégias de decisão.

Essas evoluções não deverão exigir alterações significativas na arquitetura principal do motor.

8.8 Princípios do Motor de Simulação

MS-001

O Motor de Simulação deverá utilizar exclusivamente os casos de uso da aplicação para modificar o estado do sistema.

MS-002

Nenhuma regra de negócio deverá existir apenas dentro do Motor de Simulação.

MS-003

Toda alteração produzida pela simulação deverá respeitar o Modelo de Domínio e as Regras de Negócio da aplicação.

MS-004

O estado oficial da simulação deverá ser representado pelos dados persistidos no banco de dados.

MS-005

O Motor de Simulação deverá permanecer desacoplado dos detalhes de infraestrutura.

MS-006

A arquitetura do Motor de Simulação deverá permitir evolução incremental sem comprometer a organização geral da aplicação.

9. Comunicação Interna

9.1 Objetivo

Este capítulo define como os componentes internos do Lindvior deverão se comunicar.

Seu objetivo é estabelecer diretrizes para a colaboração entre módulos, camadas e componentes da aplicação, preservando o baixo acoplamento, a alta coesão e a organização arquitetural definida neste documento.

Toda comunicação interna deverá ocorrer de forma controlada e respeitar as responsabilidades de cada componente.

9.2 Princípios Gerais

A comunicação entre componentes deverá seguir os seguintes princípios:

- respeitar as camadas da arquitetura;
 - preservar a independência dos módulos;
 - evitar dependências cíclicas;
 - utilizar contratos claramente definidos;
 - impedir acesso direto a detalhes internos de implementação.
-

9.3 Comunicação entre Camadas

As camadas deverão comunicar-se exclusivamente no sentido descendente.

Interface

|



Aplicação

|



Domínio

|



Infraestrutura

Nenhuma camada poderá acessar diretamente uma camada superior.

9.4 Comunicação entre Módulos

Sempre que um módulo necessitar utilizar funcionalidades pertencentes a outro módulo, a comunicação deverá ocorrer por meio da Camada de Aplicação.

Módulo A

|

Application Service

|

Módulo B

Os módulos não deverão acessar diretamente componentes internos uns dos outros.

9.5 Comunicação com a Persistência

O acesso ao banco de dados deverá ocorrer exclusivamente através da camada de infraestrutura.

Service

|

Repository

|

Banco de Dados

Nenhum componente pertencente às demais camadas poderá executar consultas diretamente no banco.

9.6 Comunicação do Motor de Simulação

O Motor de Simulação deverá utilizar os mesmos serviços disponibilizados para a API REST.

Simulation Engine

|

Application Service

|

Domínio

|

Repository

Essa estratégia garante que todas as regras de negócio permaneçam centralizadas no domínio.

9.7 Compartilhamento de Componentes

Componentes reutilizáveis deverão ser disponibilizados pelo módulo **Shared**.

Exemplos:

- constantes;
- utilitários;
- exceções comuns;
- validadores compartilhados;
- componentes reutilizáveis.

O módulo **Shared** não deverá conter regras de negócio.

9.8 Comunicação com Componentes Externos

Toda comunicação com recursos externos deverá ocorrer por meio da camada de infraestrutura.

Exemplos:

- banco de dados;
- autenticação;
- mensageria;
- armazenamento de arquivos;
- serviços externos.

Essa separação evita que o domínio dependa de tecnologias específicas.

9.9 Evolução da Comunicação

A estratégia de comunicação definida neste documento deverá permanecer válida durante a evolução da arquitetura.

A introdução de novos mecanismos de comunicação, como mensageria assíncrona, eventos de domínio ou microsserviços, deverá preservar os princípios estabelecidos neste capítulo.

9.10 Princípios da Comunicação

MCOM-001

Toda comunicação deverá respeitar a arquitetura em camadas.

MCOM-002

Os módulos deverão permanecer fracamente acoplados.

MCOM-003

Nenhum componente poderá acessar diretamente detalhes internos de outro módulo.

MCOM-004

Toda comunicação com recursos externos deverá ocorrer através da infraestrutura.

MCOM-005

Componentes compartilhados deverão permanecer concentrados no módulo **Shared**.

MCOM-006

A estratégia de comunicação deverá facilitar futuras evoluções arquiteturais sem comprometer a organização da aplicação.

10. Gerenciamento de Transações

10.1 Objetivo

Este capítulo define como as transações deverão ser gerenciadas na arquitetura do Lindvior.

Seu objetivo é garantir que operações que alterem o estado da aplicação sejam executadas de forma íntegra, preservando a consistência do domínio e do banco de dados.

As transações representam uma responsabilidade da arquitetura e não das regras de negócio, sendo utilizadas para assegurar que conjuntos de operações sejam concluídos integralmente ou completamente revertidos em caso de falha.

10.2 Responsabilidade pelas Transações

O gerenciamento das transações deverá ocorrer exclusivamente na Camada de Aplicação.

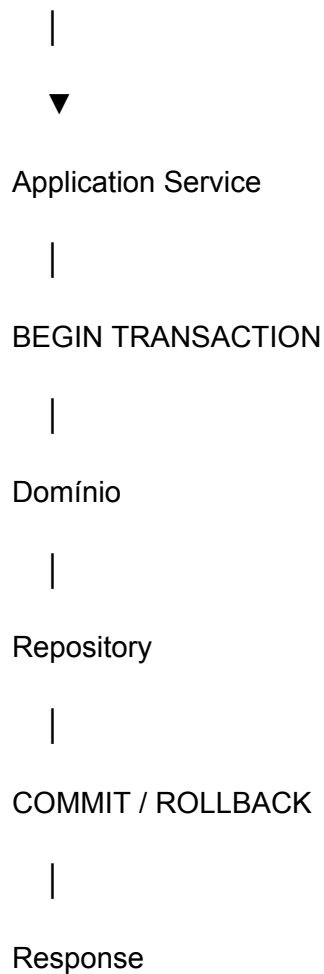
Os componentes responsáveis pelos casos de uso deverão iniciar, controlar e finalizar cada transação.

O domínio não deverá possuir conhecimento sobre mecanismos transacionais.

10.3 Fluxo Transacional

Toda operação de escrita deverá seguir o seguinte fluxo.

Controller



A transação deverá envolver toda a execução do caso de uso.

10.4 Operações Transacionais

Deverão ser executadas dentro de transações todas as operações que alterarem o estado da aplicação.

Exemplos:

- criação de usuários;
 - atualização de usuários;
 - criação de vagas;
 - atualização de vagas;
 - criação de sessões;
 - encerramento de sessões;
 - alterações administrativas.
-

10.5 Operações Não Transacionais

Operações exclusivamente de leitura não deverão iniciar transações de escrita.

Exemplos:

- consultas do dashboard;
- consultas de vagas;
- consultas de sessões;
- consultas de usuários;
- relatórios;
- indicadores.

Sempre que possível, essas operações deverão apenas consultar o estado atual da aplicação.

10.6 Tratamento de Falhas

Caso qualquer etapa de uma operação transacional falhe, todas as alterações realizadas durante aquela transação deverão ser revertidas.

A aplicação nunca deverá permanecer em estado parcialmente atualizado.

10.7 Limites das Transações

As transações deverão envolver apenas as operações necessárias para execução do caso de uso.

Operações demoradas ou que não alterem o estado da aplicação não deverão permanecer dentro da transação quando isso não for necessário.

Esse princípio reduz contenção de recursos e melhora o desempenho da aplicação.

10.8 Integração com o Motor de Simulação

As operações executadas pelo Motor de Simulação deverão utilizar exatamente a mesma estratégia transacional utilizada pelas requisições da API.

Cada evento produzido pela simulação deverá executar seu próprio caso de uso, iniciando e encerrando sua própria transação.

Isso garante que uma falha em um evento da simulação não comprometa os demais eventos executados posteriormente.

10.9 Princípios do Gerenciamento de Transações

MTX-001

Toda alteração de estado deverá ocorrer dentro de uma transação.

MTX-002

A Camada de Aplicação será a única responsável pelo gerenciamento transacional.

MTX-003

O domínio não deverá possuir dependência de mecanismos de transação.

MTX-004

Toda falha deverá provocar rollback completo da operação.

MTX-005

Operações de leitura não deverão iniciar transações de escrita.

MTX-006

As transações deverão possuir a menor duração possível, envolvendo apenas as operações necessárias para execução do caso de uso.

MTX-007

O Motor de Simulação deverá seguir exatamente a mesma estratégia transacional utilizada pelas operações da API.

MTX-008

O gerenciamento transacional deverá permanecer transparente para as regras de negócio, preservando a independência do domínio.

11. Princípios Arquiteturais

11.1 Objetivo

Este capítulo estabelece os princípios que deverão orientar todas as decisões arquiteturais do Lindvior.

Seu objetivo é garantir que a evolução da aplicação ocorra de forma consistente, preservando a organização do código, a separação de responsabilidades e a qualidade da arquitetura ao longo de todo o desenvolvimento.

Os princípios definidos neste capítulo deverão ser considerados durante a implementação de qualquer funcionalidade da aplicação.

11.2 Simplicidade

A arquitetura deverá priorizar soluções simples.

Novos componentes, padrões ou tecnologias somente deverão ser incorporados quando resolverem um problema concreto da aplicação.

Complexidade desnecessária deverá ser evitada.

11.3 Separação de Responsabilidades

Cada componente deverá possuir uma responsabilidade única e claramente definida.

Mudanças em uma responsabilidade específica deverão impactar apenas o componente responsável por ela.

Essa separação deverá ser observada em todos os níveis da aplicação:

- módulos;
 - camadas;
 - classes;
 - métodos.
-

11.4 Baixo Acoplamento

Os componentes deverão possuir o menor número possível de dependências entre si.

A comunicação deverá ocorrer exclusivamente pelos contratos definidos pela arquitetura.

Dependências diretas entre componentes internos de módulos distintos deverão ser evitadas.

11.5 Alta Coesão

Cada módulo deverá reunir funcionalidades pertencentes ao mesmo contexto de negócio.

Classes pertencentes ao mesmo módulo deverão colaborar para um objetivo comum, evitando agrupamentos artificiais de responsabilidades.

11.6 Centralização das Regras de Negócio

Todas as regras de negócio deverão permanecer concentradas na Camada de Domínio.

Controllers, Repositories, componentes de infraestrutura e utilitários não deverão implementar decisões pertencentes ao domínio.

Essa centralização garante consistência entre a API, o Motor de Simulação e futuras integrações.

11.7 Independência Tecnológica

O domínio da aplicação não deverá depender de tecnologias específicas.

Mudanças em frameworks, bibliotecas ou mecanismos de infraestrutura não deverão exigir alterações nas regras de negócio.

A arquitetura deverá preservar essa independência durante toda a evolução do projeto.

11.8 Reutilização

Sempre que possível, componentes reutilizáveis deverão ser compartilhados por toda a aplicação.

Entretanto, reutilização não deverá comprometer a clareza das responsabilidades nem aumentar o acoplamento entre módulos.

11.9 Evolução Incremental

A arquitetura deverá evoluir de forma gradual.

Novos componentes arquiteturais deverão ser incorporados apenas quando houver necessidade comprovada pelo crescimento da aplicação.

Essa estratégia permite manter a simplicidade da versão atual sem impedir evoluções futuras.

11.10 Testabilidade

A arquitetura deverá facilitar a implementação de testes automatizados.

Os componentes deverão ser organizados de forma a permitir:

- testes unitários;
- testes de integração;
- testes de arquitetura;
- testes de comportamento da simulação.

A organização arquitetural nunca deverá dificultar a validação da aplicação.

11.11 Escalabilidade Arquitetural

Embora a primeira versão seja implementada como um monólito modular, sua organização deverá permitir futuras evoluções, como:

- mensageria;
- cache distribuído;
- processamento assíncrono;
- observabilidade;
- microsserviços.

Essas evoluções deverão ocorrer preservando a organização estabelecida neste documento.

11.12 Princípios Gerais

MAP-001

A arquitetura deverá permanecer simples durante toda a evolução da aplicação.

MAP-002

Cada componente deverá possuir responsabilidade única.

MAP-003

O domínio deverá permanecer independente de tecnologias externas.

MAP-004

A comunicação entre componentes deverá respeitar as camadas da arquitetura.

MAP-005

As regras de negócio deverão permanecer centralizadas no domínio.

MAP-006

A evolução da arquitetura deverá ocorrer de forma incremental e controlada.

MAP-007

Toda nova funcionalidade deverá respeitar a organização arquitetural definida neste documento.

MAP-008

A arquitetura deverá priorizar manutenção, legibilidade e facilidade de evolução em detrimento de soluções excessivamente complexas.

12. Evolução da Arquitetura

12.1 Objetivo

Este capítulo estabelece as diretrizes para a evolução da arquitetura do Lindvior.

Seu objetivo é garantir que novas tecnologias, componentes e padrões possam ser incorporados à aplicação sem comprometer a organização arquitetural definida neste documento.

A evolução da arquitetura deverá ocorrer de forma incremental, preservando a simplicidade da solução e evitando alterações estruturais desnecessárias.

12.2 Evolução Incremental

A arquitetura deverá evoluir acompanhando as necessidades reais da aplicação.

Novos componentes arquiteturais somente deverão ser introduzidos quando resolverem problemas concretos identificados durante a evolução do sistema.

A adoção antecipada de soluções complexas deverá ser evitada.

12.3 Evolução da Infraestrutura

A organização arquitetural deverá permitir a incorporação gradual de novos recursos de infraestrutura.

Entre as evoluções previstas para o projeto encontram-se:

- containerização da aplicação;
- cache distribuído;
- mensageria;
- observabilidade;
- integração contínua;
- entrega contínua;
- processamento assíncrono;
- extração futura de serviços independentes.

A introdução desses recursos não deverá exigir alterações significativas nas regras de negócio.

12.4 Evolução dos Módulos

Novos módulos poderão ser incorporados sempre que surgirem novos contextos funcionais claramente identificados.

A criação de novos módulos deverá preservar:

- baixo acoplamento;
 - alta coesão;
 - independência entre responsabilidades;
 - organização arquitetural existente.
-

12.5 Evolução da Camada de Domínio

O domínio deverá permanecer o componente mais estável da aplicação.

Mudanças em infraestrutura ou tecnologias externas não deverão exigir alterações nas entidades, regras de negócio ou objetos pertencentes ao domínio, salvo quando houver mudança efetiva nas regras do sistema.

12.6 Preparação para Microsserviços

Embora a versão inicial seja um monólito modular, sua organização deverá facilitar futuras extrações de serviços independentes.

Essa preparação será obtida por meio de:

- módulos bem definidos;
- baixo acoplamento;
- contratos claros entre componentes;
- responsabilidades isoladas.

A migração para microsserviços somente deverá ocorrer quando houver justificativa técnica e operacional.

12.7 Compatibilidade Arquitetural

Toda evolução da arquitetura deverá permanecer compatível com os documentos oficiais do Lindvior.

Mudanças estruturais deverão preservar:

- requisitos funcionais;
- regras de negócio;
- modelo de domínio;
- modelo de dados;
- organização arquitetural.

Sempre que necessário, os documentos impactados deverão ser atualizados antes da implementação das alterações.

12.8 Princípios da Evolução

MEA-001

A arquitetura deverá evoluir de forma incremental.

MEA-002

Novos componentes arquiteturais somente deverão ser adicionados quando resolverem problemas reais da aplicação.

MEA-003

A evolução da infraestrutura não deverá comprometer o domínio da aplicação.

MEA-004

Novos módulos deverão respeitar a organização arquitetural existente.

MEA-005

Mudanças arquiteturais deverão ser previamente documentadas.

MEA-006

Toda evolução deverá preservar a simplicidade, a legibilidade e a facilidade de manutenção do sistema.

MEA-007

A preparação para futuras tecnologias não deverá introduzir complexidade desnecessária na versão atual da aplicação.

13. Considerações Gerais

13.1 Objetivo

Este capítulo apresenta as considerações finais sobre a arquitetura definida para o Lindvior.

Seu objetivo é consolidar os princípios estabelecidos ao longo deste documento e definir a arquitetura como referência oficial para toda a implementação da aplicação.

As definições apresentadas deverão orientar o desenvolvimento do sistema desde sua primeira versão até suas futuras evoluções.

13.2 Papel da Arquitetura

A arquitetura definida neste documento estabelece como os componentes do Lindvior deverão ser organizados e como deverão colaborar entre si.

Ela representa a estrutura oficial da aplicação, determinando:

- organização em camadas;
- organização modular;
- fluxo das operações;
- responsabilidades dos componentes;
- comunicação entre módulos;
- gerenciamento transacional;
- organização do Motor de Simulação.

Essas definições deverão ser respeitadas durante todo o desenvolvimento do projeto.

13.3 Integração com os Demais Documentos

O Documento 06 complementa os documentos anteriormente definidos.

Sua relação com os demais documentos pode ser resumida da seguinte forma:

- **Documento 01** — define a visão geral do projeto.
- **Documento 02** — define os requisitos funcionais.
- **Documento 03** — estabelece as regras de negócio.
- **Documento 04** — define o modelo de domínio.
- **Documento 05** — define o modelo de dados.
- **Documento 06** — define como toda essa estrutura será organizada na implementação da aplicação.

Todos os documentos deverão permanecer consistentes entre si.

13.4 Utilização Durante o Desenvolvimento

Durante a implementação do Lindvior, este documento deverá servir como referência para:

- organização dos pacotes;
- implementação das camadas da aplicação;
- definição dos módulos;
- criação dos componentes arquiteturais;
- implementação dos casos de uso;
- organização do Motor de Simulação;
- definição da comunicação entre componentes;
- gerenciamento das transações.

Sempre que houver dúvidas sobre a organização estrutural da aplicação, este documento deverá ser considerado a principal referência.

13.5 Evolução da Arquitetura

A arquitetura poderá evoluir ao longo do desenvolvimento do projeto, desde que preserve os princípios definidos neste documento.

Toda alteração arquitetural deverá:

- manter compatibilidade com os documentos oficiais;
- preservar a separação de responsabilidades;
- manter baixo acoplamento e alta coesão;
- evitar complexidade desnecessária;
- ser documentada antes de sua implementação.

A evolução deverá ocorrer de forma incremental, acompanhando as necessidades reais da aplicação.

13.6 Considerações Finais

A arquitetura definida neste documento representa a estrutura oficial da aplicação Lindvior.

Seu propósito é garantir uma implementação organizada, consistente e alinhada aos requisitos funcionais, às regras de negócio, ao modelo de domínio e ao modelo de dados definidos nos documentos anteriores.

Ao estabelecer uma arquitetura baseada em módulos, camadas e responsabilidades bem definidas, este documento reduz decisões arquiteturais durante o desenvolvimento, facilita a manutenção do código, favorece a implementação de testes automatizados e prepara a aplicação para evoluções futuras sem comprometer a simplicidade da versão inicial.

Toda implementação do Lindvior deverá utilizar este documento como referência para a organização estrutural da aplicação.